

FedMed — One-page summary FedMed — One-page summary A short summary of what has been implemented on the feat/flower-monai-scaffold branch so far and how to view this page locally. Overview FedMed is a scaffold for privacy-preserving cross-silo federated learning for medical imaging. The scaffold is designed to be ported to Flower (federated orchestration), PyTorch + MONAI (3D U-Net segmentation), and a future TenSEAL homomorphic encryption integration. Implemented features (scaffold) Feature branch: feat/flower-monai-scaffold requirements.txt listing the primary dependencies (Flower, torch, MONAI, numpy, pytest) Basic Python package layout: fedmed/ with __init__.py Model package: fedmed/model/unet3d.py (UNet3D stub with lazy PyTorch build) Flower templates: fedmed/fl/server.py and fedmed/fl/client.py (lazy imports) Encryption scaffolding: fedmed/encryption/interface.py (TenSEAL stub) and fedmed/encryption/mock.py (MockSecureAggregator) Scripts: PDF generator (fedmed/scripts/generate_pdf.py) and a local demo runner (fedmed/scripts/run_demo.py) Documentation: this one-page summary and a generated PDF (fedmed/docs/one_pager.pdf) How to open this page in your browser From Windows File Explorer: double-click fedmed/docs/one_pager.html. Or run this command from PowerShell or cmd: start "" "C:\Users\user\OneDrive\Desktop\Axlero Internship\FedMed\fedmed-federated-learning.worktrees\fedmed-federated-learning-privacy-ml\fedmed\docs\one_pager.html" Or open the file directly in most editors (VS Code: File → Open File...). How to run the code (local demo) These steps show how to run the scaffold locally. The demo is dependency-light and uses the NumPy-based mocks so it runs without PyTorch or TenSEAL. For the full stack (Flower, PyTorch, MONAI, TenSEAL) install the dependencies in requirements.txt and follow the same steps. Create and activate a virtual environment (recommended): python -m venv .venv .venv\Scripts\Activate.ps1 # PowerShell # or .venv\Scripts\activate.bat # cmd Install Python requirements (optional for full stack): pip install -r requirements.txt Regenerate the PDF one-pager if you modified the HTML: python fedmed\scripts\generate_pdf.py Run the local federated demo (pure-Python, no Flower required): python fedmed\scripts\run_demo.py The demo simulates three hospital nodes, each producing a small weight vector. Secure aggregation is simulated with pairwise masks and the recovered averaged update is printed. To run the Flower server/client templates (requires flwr): python fedmed\fl\server.py # starts a Flower server (requires flwr) python fedmed\fl\client.py # connects a minimal client to server (requires flwr) Notes and next steps The UNet3D module provides a build_torch_model() method that constructs a tiny torch.nn.Module when PyTorch is installed. This keeps imports safe for environments without torch. The TenSEAL stub and MockSecureAggregator allow developing and testing aggregation wiring before integrating native HE libraries. To port to the production stack: replace LocalTrainer.get_weights() with actual PyTorch parameter extraction and plug in TenSEAL or an MPC-based secure aggregation implementation. Branch: feat/flower-monai-scaffold. Commits will be pushed to your remote as requested.